1 Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.1 Data type of all columns in the "customers" table.

Filter Enter property name or value

Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
customer_id	STRING	NULLABLE	-	-	-	-	-
customer_unique_id	STRING	NULLABLE	-	-	-	-	-
customer_zip_code_prefix	INTEGER	NULLABLE	-	-	-	-	-
customer_city	STRING	NULLABLE	-	-	-	-	-
customer_state	STRING	NULLABLE	-	-	-	-	-

Remarks: NA

1.2 Get the time range between which the orders were placed.

```
select  min(order_purchase_timestamp) start_range, max(order_purchase_timestamp)
end_range from `Target.orders`
```

JOB INFORMATION RESULTS CHART PREVIEW JS

Row	start_range	end_range
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Remarks: the order range is from 4th September 2016 to 17th October 2018

1.3 Count the Cities & States of customers who ordered during the given period.

```
select count(distinct customer_city) as City_count,count(distinct customer_state)
as State_count from (
    select distinct customer_city,customer_state,order_purchase_timestamp,
min(order_purchase_timestamp) start_range,  max(order_purchase_timestamp) end_range
    from `Target.customers` C inner join `Target.orders` o
    on C.customer_id = o.customer_id
    group by order_purchase_timestamp,customer_city,customer_state) t
where order_purchase_timestamp between start_range and end_range
```

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	City_count	State_count
1	4119	27

Remarks: Total 4119 city from 27 states placed the order

2 In-depth Exploration:

2.1 Is there a growing trend in the no. of orders placed over the past years?

```
select count(order_id) ORDER_COUNT,order_year from (select order_id,
extract(year from order_purchase_timestamp) as Order_year
from `Target.orders`) t
group by order_year
order by order_year
```

Row	ORDER_COUNT	order_year
1	329	2016
2	45101	2017
3	54011	2018

Remaeks: The orders are growing continuously throughout range

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
select count(order_id) ORDER_COUNT,order_month,Order_year from (select order_id,
extract(Month from order_purchase_timestamp) as Order_month, extract(Year from
order_purchase_timestamp) Order_year
from `Target.orders`) t
group by  order_month,Order_year
order by  ORDER_COUNT desc
```

Row	ORDER_COUNT	order_month	Order_year
1	7544	11	2017
2	7269	1	2018
3	7211	3	2018
4	6939	4	2018
5	6873	5	2018
6	6728	2	2018
7	6512	8	2018
8	6292	7	2018
9	6167	6	2018
10	5673	12	2017
11	4631	10	2017
12	4331	8	2017
13	4285	9	2017
14	4026	7	2017
15	3700	5	2017

Remarks: though the orders are continuously increasing over the year but we need to check what went well in November 2017 and what went wrong in September and October of year 2018

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

```
SELECT COUNT(ORDER_ID) ORDER_COUNT, DAY_BREAK FROM (
    SELECT ORDER_ID, CASE
          WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
"Dawn"
          WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
"Mornings"
          WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18
THEN "Afternoon"
          WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23
THEN "Night"
        END AS DAY_BREAK
      FROM `Target.orders`) T
GROUP BY DAY_BREAK
```

Row	ORDER_COUNT	DAY_BREAK
1	27733	Mornings
2	5242	Dawn
3	38135	Afternoon
4	28331	Night

Remarks: Most of orders were made during the afternoon

3 Evolution of E-commerce orders in the Brazil region:

3.1 Get the month on month no. of orders placed in each state.

```
SELECT EXTRACT(MONTH FROM order_purchase_timestamp) AS month,EXTRACT(YEAR FROM
order_purchase_timestamp) AS year,
    customer_state,COUNT(*) AS num_orders
FROM `Target.orders` o JOIN `Target.customers` c ON o.customer_id = c.customer_id
GROUP BY month,year,customer_state
ORDER BY num_orders desc ;
```

Row	month	year	customer_state	num_orders
1	8	2018	SP	3253
2	5	2018	SP	3207
3	4	2018	SP	3059
4	1	2018	SP	3052
5	3	2018	SP	3037
6	11	2017	SP	3012
7	7	2018	SP	2777
8	6	2018	SP	2773
9	2	2018	SP	2703
10	12	2017	SP	2357
11	10	2017	SP	1793
12	8	2017	SP	1729

Remarks: State ‘SP’ is the top city to place most of the orders.

3.2 How are the customers distributed across all the states?

```
SELECT COUNT(CUSTOMER_ID) AS NUMBER_OF_CUSTOMERS, CUSTOMER_STATE FROM
`Target.customers`
GROUP BY customer_state
ORDER BY NUMBER_OF_CUSTOMERS desc
```

Row	NUMBER_OF_CUSTO	CUSTOMER_STATE
1	41746	SP
2	12852	RJ
3	11635	MG
4	5466	RS
5	5045	PR
6	3637	SC
7	3380	BA
8	2140	DF
9	2033	ES
10	2020	GO
11	1652	PE
12	1336	CE

Remarks: Most of the customers belong to 'SP'

4. **Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.**

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only)

```
SELECT
    round((SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2018 and
EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8 THEN PAYMENT_VALUE
ELSE 0 END) -
     SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2017 and
EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8 THEN PAYMENT_VALUE
ELSE 0 END)) /
    SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2017 and
EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8 THEN PAYMENT_VALUE
ELSE 0 END) * 100,2) AS `% variation`
FROM
    `Target.payments`
INNER JOIN
    `Target.orders` USING (ORDER_ID)
```

Row	% variation
1	136.98

Remarks: Cost of orders increased to approx 137% from 2017 to 2018

4.2 Calculate the Total & Average value of order price for each state.

```
select round(sum(price),2) as sum_of_price, round(avg(price),2) as AVG_of_price,
customer_state from `Target.order_items` inner join `Target.orders` using(order_id)
inner join `Target.customers`using (customer_id)
group by customer_state
order by customer_state
```

Row	sum_of_price	AVG_of_price	customer_state
1	15982.95	173.73	AC
2	80314.81	180.89	AL
3	22356.84	135.5	AM
4	13474.3	164.32	AP
5	511349.99	134.6	BA
6	227254.71	153.76	CE
7	302603.94	125.77	DF
8	275037.31	121.91	ES
9	294591.95	126.27	GO
10	119648.22	145.2	MA
11	1585308.03	120.75	MG
12	116812.64	142.63	MS

Remarks: State 'SP' has contributed towards the highest sum of orders but at the bottom in case of average.

4.3 Calculate the Total & Average value of order freight for each state.

```
select round(sum(freight_value),2) as sum_of_freight, round(avg(freight_value),2)
as AVG_of_freight, customer_state from `Target.order_items` inner join
`Target.orders` using(order_id) inner join `Target.customers`using (customer_id)
group by customer_state
order by sum_of_freight desc
```

Row	sum_of_freight	AVG_of_freight	customer_state
1	718723.07	15.15	SP
2	305589.31	20.96	RJ
3	270853.46	20.63	MG
4	135522.74	21.74	RS
5	117851.68	20.53	PR
6	100156.68	26.36	BA
7	89660.26	21.47	SC
8	59449.66	32.92	PE
9	53114.98	22.77	GO
10	50625.5	21.04	DF
11	49764.6	22.06	ES
12	48351.59	32.71	CE

5 Analysis based on sales, freight and delivery time.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query

```
select date_diff(order_delivered_customer_date,order_purchase_timestamp, day) as
`No_of_days_taken_to_deliver` ,
      date_diff(order_delivered_customer_date,order_estimated_delivery_date,day) as
`diff_in_est&act_delivery`
 from `Target.orders`
 order by order_id
```

Row	No_of_days_taken_to_deliver	diff_in_est&act_delivery
1	7	-8
2	16	-2
3	7	-13
4	6	-5
5	25	-15
6	6	-14
7	8	-16
8	5	-15
9	9	0
10	2	-18
11	4	-10
12	10	0
13	6	-21

Remarks: in column diff_in_est&act_delivery -ve value means early delivery and +ve value means late delivery compared to estimated delivery

5.2 Find out the top 5 states with the highest & lowest average freight value.

```
SELECT TOP_5_STATE,BOTTOM_5_STATE FROM (
        SELECT CUSTOMER_STATE AS `TOP_5_STATE`,TOP_5 FROM (
            SELECT customer_state,AVG_of_freight, DENSE_RANK() OVER (ORDER BY
AVG_of_freight DESC) TOP_5 FROM (
                select avg(freight_value) as AVG_of_freight, customer_state from
`Target.order_items` inner join `Target.orders` using(order_id) inner join
`Target.customers`using (customer_id)
                group by customer_state
                order by customer_state
            )
        )
        WHERE TOP_5 <= 5
        ORDER BY TOP_5) T1
INNER JOIN
        (SELECT CUSTOMER_STATE AS`BOTTOM_5_STATE`,BOTTOM_5 FROM (
            SELECT customer_state,AVG_of_freight,DENSE_RANK() OVER (ORDER BY
AVG_of_freight) BOTTOM_5 FROM (
                select avg(freight_value) as AVG_of_freight, customer_state from
`Target.order_items` inner join `Target.orders` using(order_id) inner join
`Target.customers`using (customer_id)
                group by customer_state
                order by customer_state
                )
        )
        WHERE BOTTOM_5 <= 5
        ORDER BY BOTTOM_5) T2
ON TOP_5=BOTTOM_5
```

Row	TOP_5_STATE	BOTTOM_5_STATE
1	PB	PR
2	RO	MG
3	RR	SP
4	PI	DF
5	AC	RJ

Remarks: NA

5.3 Find out the top 5 states with the highest & lowest average delivery time.

```
SELECT TOP_5_STATE,BOTTOM_5_STATE FROM (
        SELECT CUSTOMER_STATE AS `TOP_5_STATE`,TOP_5 FROM (
            SELECT customer_state,AVG_Delivery_time, DENSE_RANK() OVER (ORDER BY
AVG_Delivery_time DESC) TOP_5 FROM (
                select
avg(date_diff(order_delivered_customer_date,order_purchase_timestamp, day)) as
AVG_Delivery_time, customer_state from `Target.orders` inner join
`Target.customers`using (customer_id)
                group by customer_state
                order by customer_state
            )
        )
        WHERE TOP_5 <= 5
        ORDER BY TOP_5) T1
INNER JOIN
        (SELECT CUSTOMER_STATE AS`BOTTOM_5_STATE`,BOTTOM_5 FROM (
            SELECT customer_state,AVG_Delivery_time,DENSE_RANK() OVER (ORDER BY
AVG_Delivery_time) BOTTOM_5 FROM (
                select
avg(date_diff(order_delivered_customer_date,order_purchase_timestamp, day)) as
AVG_Delivery_time, customer_state from `Target.orders` inner join
`Target.customers`using (customer_id)
                group by customer_state
                order by customer_state
                )
        )
        WHERE BOTTOM_5 <= 5
        ORDER BY BOTTOM_5) T2
ON TOP_5=BOTTOM_5
```

Row	TOP_5_STATE	BOTTOM_5_STATE
1	AL	DF
2	AM	MG
3	RR	SP
4	AP	PR
5	PA	SC

Remarks: NA

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

```
select customer_state,delivery_time from (select customer_state,delivery_time,
dense_rank() over(order by delivery_time desc) rk from ( select customer_state,
avg(unix_seconds(order_estimated_delivery_date))-
avg(unix_seconds(order_delivered_customer_date)) delivery_time
 from `Target.orders` inner join `Target.customers`using (customer_id)
 group by customer_state)
 order by rk)
 where rk<=5
 order by rk
```

Row	customer_state	delivery_time
1	AC	1726186.733333...
2	AM	1459137.340355...
3	RO	1397421.156006...
4	AP	1339270.730465...
5	RR	1213054.686108...

```
select customer_state,AVG_Delivery_time from(
    SELECT customer_state,AVG_Delivery_time, DENSE_RANK() OVER (ORDER BY
AVG_Delivery_time DESC) TOP_5 FROM (
        select avg(
date_diff(order_estimated_delivery_date,order_delivered_customer_date,day)) as
AVG_Delivery_time, customer_state
        from `Target.orders` inner join `Target.customers`using (customer_id)
        group by customer_state
        order by customer_state
    ))
WHERE TOP_5 <= 5
ORDER BY TOP_5
```

Row	customer_state	AVG_Delivery_time
1	AC	19.76250000000...
2	RO	19.13168724279...
3	AP	18.73134328358...
4	AM	18.60689655172...
5	RR	16.41463414634...

Remarks: I have calculated the top 5 city for fastest delivery in 2 ways, feel free to chose one

6 Analysis based on the payments:

6.1 Find the month on month no. of orders placed using different payment types.

```
SELECT EXTRACT(MONTH FROM order_purchase_timestamp) AS month,EXTRACT(YEAR FROM
order_purchase_timestamp) AS year,
    payment_type,COUNT(o.order_id) AS num_orders
FROM `Target.orders` o JOIN `Target.payments` P ON o.order_id = P.order_id
GROUP BY month,year,payment_type
ORDER BY month,year,payment_type;
```

Row	month	year	payment_type	num_orders
1	1	2017	UPI	197
2	1	2017	credit_card	583
3	1	2017	debit_card	9
4	1	2017	voucher	61
5	1	2018	UPI	1518
6	1	2018	credit_card	5520
7	1	2018	debit_card	109
8	1	2018	voucher	416
9	2	2017	UPI	398
10	2	2017	credit_card	1356
11	2	2017	debit_card	13
12	2	2017	voucher	119

Remarks: Most of the orders were placed using Credit Card

6.2 Find the no. of orders placed on the basis of the payment installments that have been paid

```
select count(order_id) Number_of_orders from `Target.payments`
where payment_sequential = payment_installments
and payment_installments > 1
```

Row	Number_of_orders
1	54

Remarks: Out of all orders placed using EMI option 54 orders have all the instalments paid.